¿Sobrevive un clasificador difuso-evolutivo interpretable a la deriva de 2026? Riesgo de los commits del kernel de Linux

Does an interpretable fuzzy-evolutionary classifier survive the 2026 drift? Linux kernel commit risk

Juan Felipe Rodríguez Galindo
Maestría en Ciencias de la Información y las Comunicaciones
Universidad Distrital Francisco José de Caldas, Bogotá, Colombia
Código 20261595004 · Inteligencia Computacional · Artículo final

***Resumen*—Predecir si un commit tendrá que corregirse permite enfocar la revisión de código. En el kernel de Linux la etiqueta es explícita: el trailer `Fixes:` de un commit posterior señala al culpable. Se construyó un dataset de 612.811 commits de 2019–2026, minado del historial completo, y se comparó un clasificador de reglas difusas cuyas funciones de pertenencia y reglas ajusta un algoritmo genético con la clase mayoritaria, la regresión logística, el *gradient boosting* y una variante sin evolución. Se usó validación temporal con hueco de 90 días, AUC-PR como métrica principal y una única evaluación en 2026, el año de la etiqueta `Assisted-by` para contribuciones asistidas por IA. El modelo propuesto no alcanzó el criterio de éxito fijado de antemano: obtuvo 0,1202 de AUC-PR, el 91,1 % del *gradient boosting* (meta: 95 %), y su caída desde validación (19,5 %) fue mayor que la de la referencia (16,3 %). Aun así, triplica la prevalencia con 3–6 reglas legibles frente a miles de parámetros, y la evolución aporta un 33 % sobre la variante sin evolución. Una segunda iteración, decidida solo en validación, no mejoró el modelo.**

***Abstract*—We build a dataset of 612.811 Linux kernel commits (2019–2026) labelled by later `Fixes:` trailers and compare a genetically tuned fuzzy rule classifier with majority, logistic regression, gradient boosting and a non-evolved variant, under temporal validation and a single 2026 test. The fuzzy model missed the pre-registered goal: its AUC-PR reached 91,1 % of gradient boosting (target 95 %) and degraded more under drift, while using only 3–6 readable rules; evolution improved AUC-PR by 33 % over the non-evolved variant.**

***Palabras clave*—predicción de defectos justo a tiempo, kernel de Linux, deriva temporal, sistemas difusos, computación evolutiva, AUC-PR.**

I. Introducción

El riesgo de un cambio de software se estima desde hace dos décadas con métricas del propio cambio: cuánto toca, cuánto se difunde y cuánta experiencia tiene su autor [1], [2]. Es la predicción de defectos justo a tiempo (JIT): el aviso llega cuando el cambio aún está en revisión. El kernel de Linux recibe miles de commits por versión y tiene una etiqueta que pocos proyectos tienen: cuando un commit corrige otro, el trailer `Fixes:` nombra al culpable, sin tener que adivinarlo con `git blame` como hace SZZ [3]. En una muestra de 383 casos, el 81 % de esos trailers señalaba de verdad la causa del bug [4].

En 2026 cambió quién escribe parte de esos commits. La versión 7.0 pidió declarar con `Assisted-by:` la ayuda de asistentes de IA [5]. La 7.3 redujo la etiqueta a `Assisted-by: LLM` [6]. El 14 de octubre se eliminó, porque solo cerca del 10 % de los 18 533 commits de la 7.3 la llevaba, aunque el uso real de LLM es mucho mayor [7]. Esto tiene una consecuencia de diseño: la "era IA" no se puede definir con la etiqueta, porque que falte no quiere decir que no hubo IA. Se define por fecha, y la etiqueta queda solo para describir un subgrupo, como cota inferior.

I-A. Problema y pregunta de investigación

Problema. Clasificar cada commit del kernel como *riesgoso* si un commit posterior lo cita con `Fixes:` en 90 días o menos, usando solo información disponible en el momento de integrarlo, y con un modelo que un mantenedor pueda leer.

Pregunta. ¿Un clasificador de a lo sumo 15 reglas difusas, cuyas funciones de pertenencia y reglas se ajustan con un algoritmo evolutivo, alcanza en los commits de 2026 un AUC-PR de al menos el 95 % del mejor modelo de referencia (regresión logística o *gradient boosting*), y pierde menos AUC-PR que ese modelo al pasar de la validación (ventanas anuales 2023, 2024 y 2025, entrenando desde 2019) a la prueba (enero–julio de 2026)?

I-B. Criterio de éxito (etapa 1 de CRISP-ML(Q) [8])

Sea AP_m^V el AUC-PR medio del modelo m en las ventanas de validación y AP_m^T su AUC-PR en la prueba de 2026, ambos con intervalo de confianza del 95 % por *bootstrap*. Sean $\Delta_m = (\mathrm{AP}_m^V - \mathrm{AP}_m^T)/\mathrm{AP}_m^V$ la caída relativa, π^T la prevalencia de positivos en la prueba (el AUC-PR de la clase mayoritaria), FE el modelo propuesto y R la mejor referencia clásica en validación. El proyecto se declara exitoso si se cumplen las tres condiciones de la Tabla I. Si no se cumplen, se reporta así, sin volver a ajustar nada con la prueba.

I-C. Datos y aprobación

Los datos son públicos: el historial completo de `torvalds/linux` (GPL-2.0), minado directamente para tener una sola fuente de 2014 a 2026. El dataset de Zenodo de

Tabla I
CRITERIO DE ÉXITO, FIJADO ANTES DE VER LOS DATOS

	Condición
C1 Desempeño	$\mathrm{AP}^{T}_{FE} \geq 0{,}95\,\mathrm{AP}^{T}_{R}$
C2 Robustez	$\Delta_{FE} < \Delta_{R}$ (diferencia observada, con IC 95 %)
C3 Piso	El IC 95 % de AP^{T}_{FE} excluye π^{T}; McNemar contra cada referencia con $\alpha = 0{,}05$
Restricción	$\leq$15 reglas y $\leq$3 antecedentes por regla

Maes Bermejo *et al.* (más de 90 000 pares commit corregido–commit que corrige, CC-BY-4.0) [9] sirve para contrastar la etiqueta. Por eso no se requiere solicitud de aprobación. Los nombres y correos de los autores se reemplazan por identificadores.

II. ESTADO DEL ARTE

II-A. Método de búsqueda

La búsqueda sigue a PRISMA 2020 [10] en versión breve. El problema se descompone en tres bloques, unidos con OR por dentro y AND entre ellos:

A (commit* OR “code change*” OR “software change*” OR patch* OR “pull request*” OR “Linux kernel”) AND
B (“defect prediction” OR “bug prediction” OR “bug-introducing” OR “bug-inducing” OR “fix-inducing” OR “just-in-time” OR “change risk” OR “risk prediction” OR “defect-prone” OR buggy) AND
C (fuzzy OR “rule-based” OR evolutionary OR “genetic algorithm” OR “genetic programming” OR “search-based” OR interpretab* OR explainab*)

La ecuación Q1 usa los tres bloques. La Q2 cambia el bloque C por uno de deriva temporal (“concept drift”, drift, “temporal validation”, “time-aware”, “realistic evaluation”, “data shift”, “distribution shift”, “time travel”). Ambas se corrieron, limitadas a 2019–2026 y con los comodines expandidos a mano, en arXiv (2026-11-06) y en OpenAlex (2026-11-07, título y resumen), las dos por API pública para que la búsqueda se pueda repetir. OpenAlex, que indexa IEEE, ACM, Springer y Elsevier, reemplaza a Scopus o IEEE Xplore por falta de acceso institucional. Los clásicos anteriores a 2019 y los trabajos encontrados por citas entran como “otras fuentes”.

Los criterios se escribieron antes del tamizaje. Se incluye un trabajo si (I1) predice el riesgo de un cambio individual o estudia la validez de esa evaluación (orden temporal, latencia, ruido de la etiqueta), (I2) reporta métricas cuantitativas y (I3) describe sus datos. Se excluye si (E1) trata reparación o generación de código, (E2) trabaja solo a nivel de archivo o módulo (se admitieron clasificadores difusos y evolutivos de módulo, porque de commit no hay), (E3) es de otro dominio, (E4) es solo un dataset o (E5) no se puede verificar. Cada trabajo incluido se contrastó con su DOI, la editorial o arXiv: 35 se leyeron completos y 23 solo por resumen o fuentes secundarias. De 195 registros de OpenAlex y 38 de arXiv, 47 pasaron el tamizaje por título y resumen y 37 se incluyeron tras el texto completo, más 21 de otras fuentes. El flujo PRISMA y la matriz completa (58 filas, con la URL consultada) se adjuntan como anexo.

II-B. Síntesis por enfoques

A. Métricas de cambio y aprendizaje clásico. El enfoque fundacional usa métricas del cambio con regresión logística [1], [2]. Sigue siendo difícil de superar: con partición cronológica, una regresión logística de una sola variable (líneas añadidas) iguala o supera en AUC a DeepJIT y CC2Vec [11].

B. Aprendizaje profundo y LLM. Las redes sobre el diff y el mensaje y los modelos preentrenados con métricas expertas [12] marcan la cota alta de AUC-ROC, cerca de 0,88 en JIT-Defects4J. Tienen tres costos para este proyecto: son opacos, se evalúan sobre todo en Java y casi nunca reportan AUC-PR. En la industria, un LLM ya puntúa el riesgo de cada diff, pero su explicación (pesos de atención) convenció a pocos desarrolladores [13].

C. Interpretabilidad, reglas, difuso y evolutivo. Los profesionales piden saber por qué se marca un cambio [14]. La respuesta dominante es explicar después una caja negra: LIME en JITLine [15], reglas locales en PyExplainer [16], commits similares en IRJIT [17]. Esas explicaciones post hoc no son formalmente correctas ni estables, lo que llevó a explicadores con garantías formales [18]. A nivel de commit, lo evolutivo y lo difuso aparecen solo como piezas auxiliares: evolución diferencial para SMOTE [15], NSGA-II para los coeficientes de una regresión logística [19], búsqueda armónica para hiperparámetros [20], un algoritmo genético que pondera variables [21] y Fuzzy C-Means que pondera la atención de una red multitarea [22]. Los sistemas difusos y neuro-difusos con metaheurísticas existen, pero a nivel de módulo, sobre datos NASA, con particiones aleatorias y exactitud como métrica [23], [24]. En [23], por ejemplo, una exactitud del 91 % esconde una sensibilidad de 0,33. No se encontró un clasificador de reglas difusas para riesgo de commits.

D. Evaluación realista, deriva y latencia. Los modelos JIT se degradan con el tiempo. En Qt y OpenStack el AUC cae entre 11 y 34 puntos en un año, y reentrenar recupera parte [25]. La validación cruzada es optimista: JITBoost baja de 0,891 a 0,863 de AUC al pasar a partición temporal [26]. Desde 2023 la deriva en JIT es una línea activa: detectarla a partir de la interpretación del modelo [27], reciclar modelos antiguos [28], actualizar de forma continua, que en Ubisoft supera al reentrenamiento periódico [29], o aprender en línea con espera de la etiqueta [30]. Ninguno de estos trabajos reporta AUC-PR. La etiqueta llega tarde: el retraso mediano hasta descubrir un defecto va de 15 a 416 días según el proyecto [31], y la espera elegida cambia la medida de desempeño hasta en 25 % [32]. La evidencia más cercana a este proyecto viene del propio kernel: al pasar de un conjunto balanceado artificialmente a la prevalencia real, el PR-AUC de ocho técnicas JIT de vulnerabilidades cae de 0,805 a 0,016 [33]. Sobre la era de la IA, los PR de agentes no inducen más bugs que los humanos en un grado relevante, aunque sí cambian el tamaño y el tipo de los cambios [34]. Es decir, la deriva esperada está en las covariables más que en la tasa de defectos.

E. La etiqueta. SZZ es ruidoso: cerca de la mitad de las

correcciones que identifica no lo son [35]; filtrarlas con un LLM reduce 21 % los commits inductores en PostgreSQL y hace la tarea más difícil de lo reportado [36]. Sus variantes alcanzan F1 de 0,49–0,51 contra la verdad de `Fixes:` en Linux [4]. Los defectos latentes que aún no se han descubierto bajan la sensibilidad cerca de 90 % si no se recorta la cola reciente [37].

II-C. Brecha

Los enfoques anteriores dejan cuatro vacíos que este proyecto atiende a la vez:

1) No hay un clasificador de reglas difusas ajustado con evolución para riesgo a nivel de commit. A ese nivel, lo difuso y lo evolutivo son auxiliares; como clasificador, se quedaron en módulos con particiones aleatorias.
2) La deriva en JIT está demostrada y se estudia activamente [25], [27], [28], pero no se ha medido con AUC-PR y prevalencia real en commits del kernel etiquetados con `Fixes:`.
3) Nadie ha medido si la interpretabilidad intrínseca (pocas reglas legibles) cuesta desempeño o, al contrario, da robustez frente a la deriva.
4) No hay evaluación en 2026, el primer año con contribuciones asistidas por IA declaradas en el kernel, donde la etiqueta misma resultó poco fiable [7].

III. Metodología: datos y modelos

La metodología sigue las etapas de CRISP-ML(Q) [8]. Esta sección cubre la comprensión y preparación de los datos (etapa 2) y la elección de los modelos (etapa 3). Las subsecciones finales declaran el protocolo de evaluación (etapa 3), fijado antes de entrenar el modelo propuesto.

III-A. Construcción del dataset

Se minó un clon completo de `torvalds/linux` con una sola pasada de `git log` sobre la rama principal, sin commits de fusión, desde 2014. Los años 2014–2018 solo sirven para calcular la experiencia de los autores. Por cada commit se extraen el *numstat* del diff, los trailers del mensaje y las fechas. Las referencias `Fixes:` (prefijos de al menos 8 caracteres hexadecimales) se resuelven contra el historial: el 9,4 % que no se resuelve apunta a commits anteriores a 2014 o a otros árboles.

Etiqueta. Un commit es *riesgoso* si algún commit posterior de la rama principal lo cita con `Fixes:` en 90 días o menos. Los commits con menos de 90 días de observación a la fecha de corte (6 de noviembre de 2026) quedan *censurados* y se excluyen (26.133 commits), porque su etiqueta negativa todavía no es definitiva [31], [37].

Anonimización. Nombre y correo del autor se reemplazan por un SHA-256 con sal. No se publica el texto de los mensajes.

La Tabla II resume la ficha del dataset y su validación con los criterios D1–D7 del enunciado. Las 22 variables candidatas son las familias clásicas de JIT [2]:

- tamaño: líneas añadidas y borradas, archivos y binarios;
- difusión: directorios de primer nivel y subsistemas;
- revisión: `Reviewed-by`, `Acked-by`, `Tested-by`, `Reported-by`, `Signed-off-by`, `Co-developed-by` y `Link`;
- propósito: si trae su propio `Fixes:`, si pide backport a *stable*, si el asunto menciona *fix* y la longitud del asunto;
- experiencia: commits previos del autor, en total y en el subsistema, y días desde su primer commit;
- momento: hora y día de la semana.

Todas se conocen al integrar el commit.

Tabla II
Ficha del dataset y verificación D1–D7

	Valor
Fuente	Clon de `torvalds/linux`, rama principal, GPL-2.0
Periodo	2019-01-01 a 2026-07-31 (corte de minería: 2026-11-06)
D1 Tamaño	612.811 commits no censurados: 558.438 de desarrollo (2019–2025) y 54.373 de prueba (2026)
D2 Objetivo	Riesgoso (binaria): 23.735 positivos en desarrollo, prevalencia anual entre 3,9 % y 4,5 %
D3 Calidad	Sin faltantes ni hashes duplicados; colas pesadas (hasta 730 092 líneas en un commit); 3 fechas de autor corruptas
D4 Estructura	Eventos irregulares ordenados por fecha del committer; 18.127 autores y 748 subsistemas
D5 Disponibilidad	Solo información del propio commit y del pasado del autor
D6 Licencia y ética	Metadatos públicos; autores anonimizados
D7 Documentación	Diccionario de las 22 variables en el anexo

III-B. Análisis exploratorio

Hay tres hallazgos que guían el diseño:

- **La prevalencia depende de la adopción de `Fixes:`.** Antes de 2019 crece año a año (Fig. 1), porque cada vez más correcciones usan el trailer, no porque el código empeore. Desde 2019 se estabiliza entre 3,9 % y 4,5 %, y por eso el entrenamiento empieza el 1 de enero de 2019. En 2026 baja a 3,39 %.
- **El tamaño es la señal dominante y no es lineal.** La prevalencia pasa de 0,9 % en el quintil más pequeño de líneas añadidas a 11,7 % en el más grande, con una distribución de colas muy pesadas.
- **Hay deriva en las covariables antes de 2026.** El número medio de `Reviewed-by` por commit sube de 0,42 (2019) a 0,71 (2025), y la fracción de commits que son correcciones, de 14,5 % a 18,4 %.

En correlación de Spearman, dos pares son casi redundantes: directorios de primer nivel y subsistemas (0,97), y experiencia y antigüedad del autor (0,73). Por eso el modelo difuso usa solo uno de cada par.

III-C. Modelos y su papel

La Tabla III resume los modelos. Las dos referencias clásicas cubren dos extremos. La regresión logística es el estándar de JIT y es difícil de superar con partición temporal

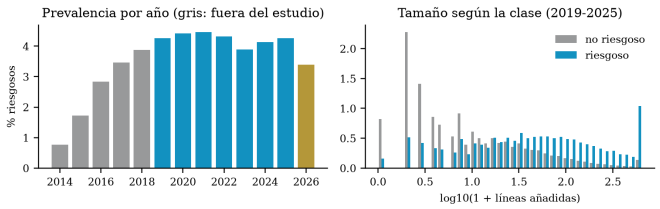


Figura 1. Izquierda: prevalencia de commits riesgosos por año (en gris, los años fuera del estudio). Derecha: distribución del tamaño por clase en 2019–2025.

Tabla III
MODELOS COMPARADOS

Modelo	Papel	Justificación
Clase mayoritaria	Trivial	Su AUC-PR es la prevalencia: el piso
Reg. logística	Clásica	Estándar de JIT [2]; $\log(1+x)$ y estandarización
Gradient boosting	Clásica fuerte	Cota alta sin restricción de interpretabilidad [38]
Difuso-evolutivo	Propuesto	$\leq$15 reglas legibles; vértices y reglas ajustados con un AG
Difuso sin evolución	Variante de control	Mismas reglas iniciales, sin AG: mide el aporte de la evolución

[2], [11]. El *gradient boosting* [38] da la cota alta de una caja negra sobre las mismas variables.

Qué propiedad de los datos aprovecha el modelo propuesto. El riesgo crece por umbrales con el tamaño y la difusión, se modula con la revisión y la experiencia, y las variables tienen colas muy pesadas. Una partición difusa sobre la escala de cuantiles captura esos umbrales sin depender de la escala. Las reglas con pocos antecedentes capturan las interacciones que un mantenedor reconoce, por ejemplo "si el cambio es grande y toca varios subsistemas". El modelo usa 10 variables fijadas antes de ver resultados: líneas añadidas y borradas, archivos, subsistemas, `Reviewed-by`, `Signed-off-by`, si es corrección, si va a *stable* y la experiencia del autor, en total y en el subsistema.

III-D. Clasificador difuso-evolutivo

Cada variable x_j se lleva a $z_j \in [0,1]$ con su función de distribución empírica, estimada solo con el entrenamiento. Sobre z_j se definen tres términos (bajo, medio, alto) que forman una partición de Ruspini [39] con vértices $a_j < b_j < c_j$: bajo vale 1 hasta a_j y cae a 0 en b_j; medio es el triángulo (a_j, b_j, c_j); alto sube de 0 en b_j a 1 en c_j. Una regla r tiene hasta tres antecedentes y un peso $w_r \in [0,1]$:

$$\mu_r(\mathbf{x}) = \prod_{(j,t)\in A_r} \mu_{jt}(z_j), \qquad s(\mathbf{x}) = 1 - \prod_r \big(1 - w_r\,\mu_r(\mathbf{x})\big), \tag{1}$$

es decir, t-norma producto y O probabilístico entre reglas. El puntaje s ordena los commits por riesgo.

El ajuste sigue el enfoque Pittsburgh de los sistemas difusos genéticos [40], [41]: cada individuo codifica todos los vértices y la base completa de reglas (activación, antecedentes, términos y pesos). La aptitud es el AUC-PR de entrenamiento, que es la métrica adecuada con clases desbalanceadas [42], menos una penalización por el número de antecedentes. Los operadores son:

- selección por torneo y elitismo;
- cruce uniforme, que hereda cada regla completa;
- mutación gaussiana de vértices y pesos, y mutación aleatoria de antecedentes y términos.

La variante sin evolución fija los vértices en los cuantiles (0,2; 0,5; 0,8) y elige reglas de 1 o 2 antecedentes de forma voraz, por su confianza difusa, mientras mejore el AUC-PR. Esa misma solución siembra la población inicial del AG. Así, la diferencia entre ambas variantes aísla el aporte de la evolución.

III-E. Pipeline sin fuga de información

Todo lo que se aprende de los datos se ajusta solo con el entrenamiento de cada ventana y se aplica sin cambios a la validación o a la prueba:

- la transformación $\log(1+x)$ y la estandarización de la regresión logística, dentro de un `Pipeline`;
- la función de distribución empírica del modelo difuso;
- el umbral de decisión.

No se balancean las clases: el AUC-PR se mide con la prevalencia real, y los conjuntos balanceados de forma artificial inflan el desempeño de manera engañosa [33]. Las variables solo usan información anterior a la integración del commit. La experiencia del autor se cuenta con los commits previos en orden temporal.

III-F. Partición temporal

Nunca se mezclan commits al azar (Fig. 2). Para cada año de validación $V \in \{2023, 2024, 2025\}$ se entrena con [2019-01-01, inicio(V) − 90 d) y se valida con el año V completo. El hueco de 90 días replica lo que ocurre en la práctica: al entrenar, las etiquetas de los últimos 90 días todavía no existen. El modelo final se entrena con [2019-01-01, 2025-10-03) y se evalúa una sola vez con los 54.373 commits de enero a julio de 2026 (prevalencia 3,46 %).

La partición de prueba quedó guardada con una huella SHA-256 de sus hashes y etiquetas. El script de evaluación final se niega a correr si la huella cambia. Se declara una salvedad: durante la compuerta de factibilidad (6 de noviembre), modelos simples se evaluaron una vez sobre 2026 para confirmar que había señal. Ninguna decisión posterior usó esas cifras.

III-G. Métricas y pruebas estadísticas

Métrica principal: AUC-PR (precisión promedio de la clase riesgoso). Con una prevalencia cercana al 4 %, el AUC-ROC es engañosamente alto y el AUC-PR muestra la utilidad real de la lista de commits marcados [42]. El AUC-PR de un clasificador sin información es igual a la prevalencia; por eso también se reporta el cociente AUC-PR/prevalencia.

Métricas secundarias: MCC, AUC-ROC, sensibilidad y especificidad. Las métricas que dependen de un umbral usan

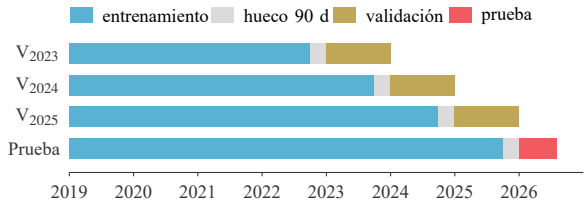


Figura 2. Partición temporal con ventana creciente. Cada ventana de validación (2023, 2024 y 2025) se entrena con todo lo anterior menos un hueco de 90 días; la prueba (enero–julio de 2026) se evalúa una sola vez.

el que maximiza el MCC en la ventana de validación 2025, aplicado tal cual en la prueba.

Costo: tiempo de entrenamiento, tiempo de inferencia y número de parámetros.

Incertidumbre. Se reportan intervalos de confianza del 95 % por *bootstrap* (1000 remuestreos de la prueba) y media ± desviación en 5 semillas (11, 23, 37, 41 y 53) para los modelos estocásticos.

Pruebas.

- McNemar con corrección de continuidad, entre el difuso-evolutivo y cada referencia, sobre las mismas predicciones de prueba [43].
- Friedman con *post hoc* de Nemenyi sobre los rangos de AUC-PR de los cinco modelos en las tres ventanas de validación más la prueba [44].

En ambos casos $\alpha = 0{,}05$. Con solo 5 semillas no se usa Wilcoxon. Una diferencia sin prueba se reporta como "diferencia observada".

III-H. Resultados de validación de las referencias

Las referencias se ajustaron con el mismo pipeline y la misma partición que el modelo propuesto. Se probó una pequeña rejilla (10 configuraciones) y se eligió la de mayor AUC-PR medio en las tres ventanas.

Mejor referencia clásica: el *gradient boosting*, con AUC-PR medio de 0,1578 y alrededor de 5.645 parámetros (nodos de los árboles). Es la referencia R del criterio de éxito.

Regresión logística: da 0,1486 con 23 parámetros, y la regularización no cambia nada porque hay cientos de miles de ejemplos.

Clase mayoritaria: queda en 0,0410, la prevalencia. Ambas referencias clásicas la multiplican por cerca de cuatro.

III-I. Espacio de hiperparámetros del modelo propuesto

El espacio se declara antes de buscar (Tabla IV). La búsqueda es aleatoria, con 14 configuraciones de las 972 posibles. Cada configuración se evalúa en las tres ventanas con la semilla 11 y se elige por el AUC-PR medio. La sensibilidad se analiza sobre la rejilla de máximo de reglas × probabilidad de mutación, con el resto fijo en la mejor configuración. Después, la configuración elegida se entrena con las 5 semillas.

Tabla IV
ESPACIO DE HIPERPARÁMETROS DEL DIFUSO-EVOLUTIVO

Hiperparámetro	Valores
Máximo de reglas	5, 10, 15
Tamaño de población	40, 60, 80
Generaciones	60, 100, 140
Prob. de mutación	0,05, 0,15, 0,3
Prob. de cruce	0,7, 0,9
λ (penalización)	0,0, 0,0005
Términos por variable	3 (Ruspini)
Antecedentes por regla	≤ 3
Torneo / elitismo	3 / 2
Muestra para la aptitud	80.000 commits

IV. EXPERIMENTACIÓN

Esta sección cubre el entrenamiento (fase F6). Todo se hizo sobre las ventanas de validación 2023–2025; la prueba de 2026 no interviene.

IV-A. Registro de experimentos

El registro de experimentos (archivo `registro_experimentos.csv`) guarda, de cada entrenamiento, la etapa, el modelo, la configuración, la semilla, la ventana, el tamaño de entrenamiento, el AUC-PR y el AUC-ROC de validación, el tiempo y el número de parámetros. Hay 30 corridas de las referencias y 84 del modelo propuesto:

- 42 de la búsqueda;
- 24 de la sensibilidad;
- 15 de las semillas;
- 3 de la variante sin evolución.

Los entrenamientos corren en paralelo (6 procesos). Uno del difuso-evolutivo con la configuración final tarda unos 11 minutos con unos 450 000 commits, porque la aptitud se evalúa sobre una muestra de 80 000.

IV-B. Búsqueda aleatoria

Se evaluaron 14 configuraciones del espacio de la Tabla IV en las tres ventanas. La mejor tiene la población y el número de generaciones más grandes del espacio y la mutación más baja, y alcanza un AUC-PR medio de 0,1485. Que gane el extremo del espacio sugiere que el AG aún tenía margen. No se amplió la búsqueda: se registra como limitación.

IV-C. Análisis de sensibilidad

Con el resto fijo en la mejor configuración, se varió el máximo de reglas y la probabilidad de mutación (Fig. 3).

La mutación es lo que más pesa. Con 0,05 el AUC-PR medio es 0,1486; con 0,30 baja a 0,1347. Una mutación alta destruye las reglas buenas antes de que se propaguen.

El máximo de reglas casi no importa. El mejor punto (10 reglas, mutación 0,05) da 0,1498, prácticamente igual que con 5 o 15. El peor es 15 reglas con mutación 0,30, con 0,1305. Esto es coherente con lo que el AG hace en la práctica: aunque se le permitan 15 reglas, termina usando entre 3 y 6.

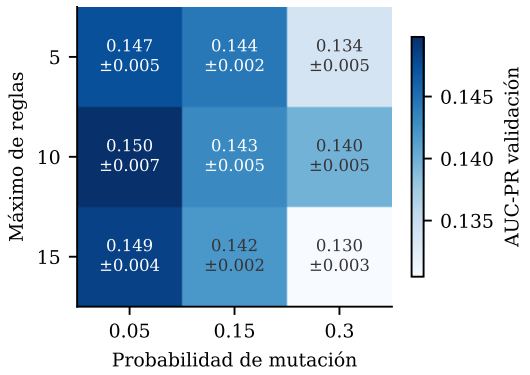


Figura 3. Sensibilidad: AUC-PR medio ± desviación en las tres ventanas, según el máximo de reglas y la probabilidad de mutación.

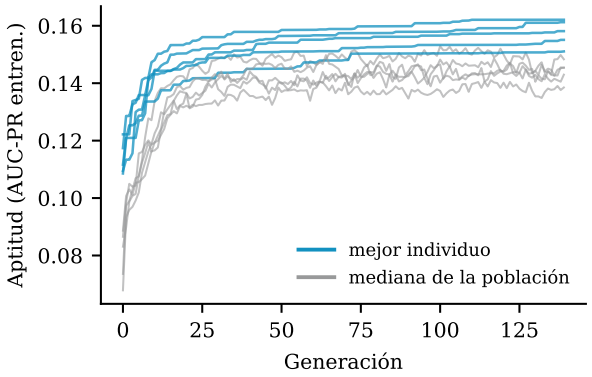


Figura 4. Curva de aprendizaje del AG (ventana 2025, 5 semillas): aptitud del mejor individuo y mediana de la población.

La configuración congelada es la de la búsqueda (15 reglas como máximo). No se cambió a 10 después de ver la sensibilidad, porque la diferencia (0,001) está dentro de la variación entre semillas.

IV-D. Convergencia

La Fig. 4 muestra la aptitud del mejor individuo y la mediana de la población, por generación, en la ventana 2025 para las 5 semillas. La mejora es rápida en las primeras 20 generaciones y se estabiliza entre la 60 y la 80. La distancia entre el mejor individuo y la mediana se mantiene, así que la población conserva diversidad hasta el final.

IV-E. Semillas y aporte de la evolución

La configuración elegida se entrenó con las 5 semillas (Tabla V). El AUC-PR medio de validación es 0,1495 ± 0,0053. Es la diferencia observada respecto a las referencias, sin prueba de significancia todavía:

- prácticamente igual a la regresión logística (0,1486);
- el 94,8 % del *gradient boosting* (0,1578);
- muy por encima de la prevalencia.

La variante sin evolución arranca de la misma solución voraz y se queda en 0,1206. Usa 15 reglas con 29 antecedentes. El AG no solo mejora el AUC-PR: también simplifica el modelo a entre 3 y 6 reglas, con 3 a 10 antecedentes en total.

IV-F. Configuración congelada

El 2026-11-07 23:41 se congeló la configuración final, antes de evaluar la prueba (`config/configuracion_final.json`). Contiene:

- los hiperparámetros del difuso-evolutivo y las 5 semillas;
- el umbral de MCC de cada semilla en la ventana 2025;
- la semilla de referencia para McNemar (11, la de AUC-PR mediano);
- la configuración y el umbral de cada referencia, con R = *gradient boosting*;
- la huella SHA-256 de la partición de prueba (`11e7039f3cc27912…`).

El modelo final se entrena con todo 2019 hasta el 3 de octubre de 2025 y se evalúa una sola vez en 2026.

V. Resultados en la prueba de 2026

La configuración congelada se evaluó una sola vez sobre los 54.373 commits de enero a julio de 2026 (prevalencia 3,46 %), tras verificar la huella de la partición. La Tabla VI reúne desempeño y costo.

V-A. Criterio de éxito

El proyecto **no alcanza** el criterio fijado en la Entrega 1:

- **C1, desempeño: no se cumple.** El difuso-evolutivo logra 0,1202 ± 0,0024 de AUC-PR, el 91,1 % del *gradient boosting* (0,1320). La meta era 95 %.
- **C2, robustez: no se cumple.** Pierde el 19,5 % de su AUC-PR de validación; el *gradient boosting* pierde el 16,3 % y la regresión logística el 14,2 %.
- **C3, piso: se cumple solo en su primera parte.** El IC 95 % del AUC-PR, [0,108; 0,128], excluye la prevalencia, y el modelo multiplica el AUC-PR del azar por cerca de 3,5. Las pruebas de McNemar son significativas ($p < 10^{-6}$) contra todas las referencias, pero a favor de ellas, salvo contra la variante sin evolución:
  - frente al *gradient boosting*, el difuso acierta donde aquel falla en 584 commits y falla donde aquel acierta en 4.379;
  - frente a la variante sin evolución, las cifras se invierten: 3.306 frente a 878.

Friedman–Nemenyi. La prueba de Friedman sobre los cuatro bloques (tres ventanas y la prueba) rechaza la igualdad de los cinco modelos ($p = 0{,}0043$). Los rangos medios son:

- *gradient boosting*: 1,00;
- difuso-evolutivo y regresión logística: 2,50;
- variante sin evolución: 4,00;
- clase mayoritaria: 5,00.

Con la diferencia crítica de Nemenyi (3,05), solo la distancia entre el *gradient boosting* y la clase mayoritaria es significativa. Con cuatro bloques, la prueba tiene poca potencia.

Tabla V
DIFUSO-EVOLUTIVO CON 5 SEMILLAS Y VARIANTE SIN EVOLUCIÓN (VALIDACIÓN)

Semilla	2023	2024	2025	Media	Reglas	Antec.	Tiempo (s)
11	0,1433	0,1485	0,1537	0,1485	4–4	5–7	685
23	0,1436	0,1449	0,1566	0,1484	3–5	5–7	694
37	0,1559	0,1469	0,1518	0,1516	4–5	6–10	690
41	0,1423	0,1440	0,1543	0,1469	3–5	3–8	665
53	0,1510	0,1462	0,1587	0,1520	4–6	5–9	393
Media ± DE	0,1472 ± 0,0053	0,1461 ± 0,0016	0,1550 ± 0,0024	0,1495			
Sin evolución	0,1201	0,1180	0,1236	0,1206	15	29	2

Tabla VI
PRUEBA DE 2026: DESEMPEÑO (MEDIA DE LAS SEMILLAS; IC 95 % POR *BOOTSTRAP* DE LA SEMILLA DE REFERENCIA) Y COSTO. MÉTRICAS CON UMBRAL: EL QUE MAXIMIZA EL MCC EN 2025

Modelo	AUC-PR	IC 95 %	AUC-ROC	MCC	Sensib.	Especif.	Parám.	Entren. (s)	ms/1000
Clase mayoritaria	0,0346	[0,033; 0,036]	0,500	0,000	0,000	1,000	1	0	0,15
Reg. logística	0,1274	[0,117; 0,140]	0,765	0,179	0,433	0,889	23	2	0,35
Gradient boosting	0,1320 ± 0,0007	[0,121; 0,146]	0,769	0,186	0,395	0,911	5.765	18	6,98
Difuso sin evolución	0,0901	[0,083; 0,098]	0,741	0,153	0,565	0,785	71	2	1,03
Difuso-evolutivo	0,1202 ± 0,0024	[0,108; 0,128]	0,759	0,132	0,278	0,928	27	581	1,24

V-B. El umbral no se transfiere

El AUC-PR del difuso-evolutivo es estable entre semillas (0,1172–0,1245). Sus métricas con umbral, en cambio, no lo son:

- el umbral fijado en 2025 marca entre el 0,0 % y el 17,9 % de los commits de 2026, según la semilla;
- el MCC varía entre 0,000 y 0,175.

La causa es estructural. Con 3 a 6 reglas, el puntaje toma pocos valores distintos, y un umbral elegido en una meseta de 2025 queda al borde de un escalón en 2026. Con la semilla de referencia marca el 17,9 %, el doble que el *gradient boosting*, y eso explica el McNemar desfavorable.

Las referencias, de puntaje continuo, conservan su tasa de marcado. Para ordenar commits por riesgo el modelo difuso sirve; para decidir con un corte fijo, su umbral debe recalibrarse con datos recientes.

V-C. Robustez y análisis de errores

Degradación dentro de 2026 (Fig. 6). Todos los modelos se degradan a lo largo del año. El difuso-evolutivo baja de 0,151 en enero a 0,086 en julio, y el *gradient boosting* de 0,158 a 0,104. La deriva crece a medida que la prueba se aleja del entrenamiento.

Subgrupo `Assisted-by` (2.031 commits, prevalencia 2,81 %). Es descriptivo, porque la etiqueta es una cota inferior. Todos los modelos rinden peor que en el resto:

- difuso-evolutivo: 0,055 frente a 0,120;
- *gradient boosting*: 0,067 frente a 0,136;
- regresión logística: 0,079 frente a 0,130.

Las señales de tamaño y difusión que funcionan en general predicen menos en los commits declarados como asistidos.

Errores por tipo de commit. Los errores del difuso se concentran en dos grupos:

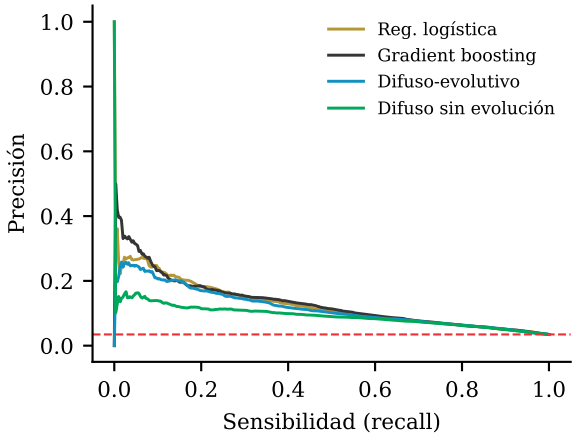


Figura 5. Curvas de precisión-sensibilidad en la prueba de 2026 (semilla de referencia). La línea roja es la prevalencia.

- **Commits pequeños.** En los tres quintiles inferiores de líneas añadidas la sensibilidad es 0: las reglas nunca los marcan. En el quintil superior (prevalencia 9,6 %) detecta el 86,8 % de los riesgosos.
- **Correcciones.** En los commits que traen su propio `Fixes:`, el AUC-PR es 0,088 frente a 0,126, y la sensibilidad 20,3 % frente a 58,7 %. Una corrección que introduce otro error suele ser pequeña.

V-D. Interpretabilidad: las reglas aprendidas

El modelo de la semilla de referencia tiene 5 reglas (Tabla VII) y 27 parámetros en promedio entre semillas, frente a 5.765 del *gradient boosting*. Cada término se resume con su corte $\alpha = 0{,}5$ en unidades originales.

Las reglas se leen solas:

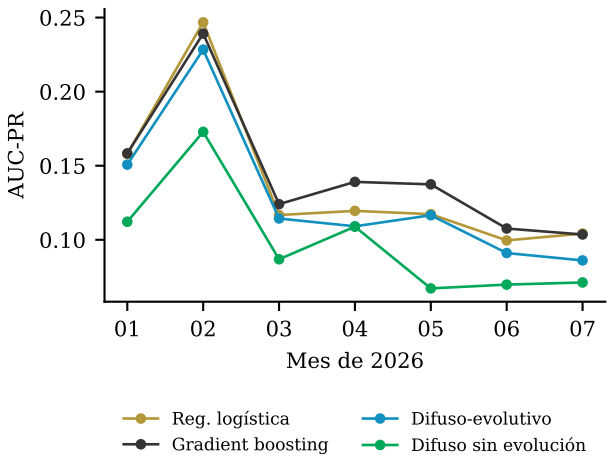


Figura 6. AUC-PR por mes de 2026 (semilla de referencia).

Tabla VII
BASE DE REGLAS DEL DIFUSO-EVOLUTIVO (SEMILLA DE REFERENCIA). ENTRE PARÉNTESIS, EL CORTE $\alpha = 0{,}5$ DE CADA TÉRMINO

	Peso	SI … ENTONCES riesgoso
R1	0,62	líneas añadidas es alto (≥16)
R2	0,29	líneas añadidas es alto (≥16) Y archivos es alto (≥2)
R3	0,21	subsistemas es medio (=1) Y líneas borradas es bajo (=0)
R4	0,13	subsistemas es alto (≥2) Y líneas borradas es medio (1–4)
R5	0,12	subsistemas es alto (≥2)

- un cambio que agrega muchas líneas es sospechoso (R1), más aún si toca varios archivos (R2);
- también lo es el que toca más de un subsistema (R4, R5);
- con menos peso, el código nuevo puro: un solo subsistema y nada borrado (R3).

Coinciden con las familias de tamaño y difusión de la literatura JIT [2]. En esta semilla, la revisión (`Reviewed-by`) y la experiencia del autor no sobrevivieron a la evolución.

Aporte de la evolución. En la prueba, la evolución sube el AUC-PR un 33 % frente a la variante sin evolución (0,1202 frente a 0,0901), y reduce el modelo de 15 reglas a 3–6.

VI. Iteración 2: vuelta al modelado

CRISP-ML(Q) no es lineal: si la evaluación no alcanza el criterio, se vuelve a la preparación de datos o al modelado. Antes de correr nada se fijó un protocolo con el candidato, la regla de selección y la regla de adopción. El diagnóstico usa solo la validación y la exploración:

- la deriva de las covariables (más `Reviewed-by` y más correcciones con el tiempo) sugiere entrenar con una ventana deslizante de 3 años;
- el difuso ve 10 de las 22 variables que usan las referencias, lo que sugiere un conjunto ampliado de 19.

El diseño es un factorial 2×2 evaluado en validación. Cuando cambia la ventana, el *gradient boosting* se reentrena con esa misma ventana.

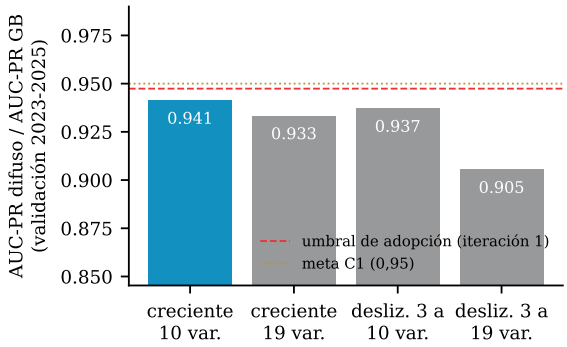


Figura 7. Iteración 2: razón de AUC-PR difuso/*gradient boosting* en validación para los cuatro candidatos (semilla 11), frente al umbral de adopción.

Una regla de adopción evita usar la prueba otra vez sin necesidad: la iteración 2 solo se evaluaría en 2026 si su razón AUC-PR$_{FE}$/AUC-PR$_{GB}$ en validación superara la de la iteración 1 (0,947).

Resultado (Fig. 7): ningún candidato la supera.

- ventana creciente con 10 variables (la iteración 1): 0,941;
- ventana deslizante con 10 variables: 0,937;
- ventana creciente con 19 variables: 0,933;
- ventana deslizante con 19 variables: 0,905.

Más variables no ayudan: el AG sigue eligiendo 3 o 4 reglas de tamaño y difusión. Una ventana más corta tampoco cambia nada, y el *gradient boosting* queda igual con ella.

La iteración 2 no se adoptó y la prueba no se volvió a usar. Las cifras de la Sección V son las definitivas.

VII. Discusión y limitaciones

El costo de la interpretabilidad es medible y moderado. Con 3 a 6 reglas legibles y unos 27 parámetros, el modelo difuso conserva el 91,1 % del AUC-PR de un *gradient boosting* con miles de nodos, y empata en rango medio con la regresión logística. La hipótesis de que la interpretabilidad intrínseca daría además robustez a la deriva no se sostiene: el difuso cae más que las referencias.

Una explicación consistente con los errores es que las reglas apuestan casi todo al tamaño y a la difusión. Cuando la deriva cambia la mezcla de commits (más correcciones pequeñas, más revisión), un modelo con pocas reglas no tiene con qué compensar.

Amenazas a la validez.

- **La prueba no estuvo intacta.** En la compuerta de factibilidad (6 de noviembre) se miró 2026 con modelos simples. Ninguna decisión se basó en eso, pero la prueba no es virgen. La iteración 2 se decidió sin volver a evaluarla.
- **La etiqueta.** `Fixes:` es fiable en torno al 81 % [4]. La censura de 90 días ignora los errores que se descubren más tarde [37].
- **El subgrupo `Assisted-by`** es una cota inferior: el 90 % de los commits asistidos no la declaraba [7].
- **Validez externa.** Se usó un solo proyecto.

- **La búsqueda.** El mejor punto quedó en el extremo del espacio (población y generaciones máximas), así que podría haber margen.
- **La iteración 2** comparó la semilla 11 contra un umbral calculado con 5 semillas. Aun con la semilla 11, ningún candidato superó a la iteración 1, así que la decisión no depende de esa asimetría.
- **McNemar** compara aciertos con un umbral fijo y, con este desbalance, castiga a quien marca más. Por eso se complementa con el AUC-PR y sus intervalos.

VIII. Conclusiones

1) El clasificador difuso-evolutivo no alcanzó el criterio de éxito. En 2026 logra 0,1202 de AUC-PR (91,1 % del *gradient boosting*, meta 95 %) y cae 19,5 % desde validación, frente a 16,3 % de la referencia.
2) Aun así, supera con amplio margen al azar (IC [0,108; 0,128] frente a una prevalencia de 3,46 %). Lo hace con 5 reglas que un mantenedor puede leer, frente a miles de parámetros.
3) La evolución es lo que hace útil al modelo: 33 % más de AUC-PR que la variante sin evolución y, a la vez, de 15 a 3–6 reglas.
4) La debilidad práctica es el umbral. Con pocas reglas el puntaje es escalonado, y un corte fijado en 2025 no se transfiere a 2026. Recalibrarlo con datos recientes es el siguiente paso natural, junto con aptitudes que premien la estabilidad temporal.
5) Ninguno de los modelos predice bien los commits declarados como asistidos por IA (AUC-PR 0,055–0,079), una pregunta abierta para cuando haya una etiqueta fiable.

Declaración de uso de IA generativa

Se usó Claude (Anthropic), mediante Claude Code, como apoyo para: escribir y ejecutar los scripts de búsqueda bibliográfica, de minería del kernel, de exploración, de los modelos y de la evaluación; proponer el tamizaje según criterios fijados de antemano; verificar con agentes web cada referencia contra su fuente; y redactar borradores del texto. Las cifras se generan desde los archivos de resultados con un script, no se transcriben a mano. El criterio de éxito, el protocolo de evaluación y el de la iteración 2 quedaron fechados en archivos de configuración antes de ver los resultados correspondientes. **[Completar: qué revisó y decidió el autor por su cuenta y cómo.]** El autor responde por cada cifra, referencia y decisión.

Anexo: reproducibilidad

Código, cuaderno ejecutado, registro de experimentos y configuración congelada se entregan con el artículo. Entorno: Python 3.13, pandas 3.1.1, scikit-learn 1.9.1, numpy 2.5.4, scipy 1.18.1. Semillas: 11, 23, 37, 41, 53 (modelos), 2026 (búsqueda), 1 y 2 (*bootstrap*). El cuaderno de Colab parte de la tabla anonimizada, verifica la huella SHA-256 de la prueba (`11e7039f3cc27912…`) y reproduce las cifras de la Tabla VI. **[Enlace público de la tabla de datos.]**

Referencias

[1] A. Mockus and D. M. Weiss, "Predicting risk of software changes," Bell Labs Tech. J., vol. 5, no. 2, pp. 169–180, Apr.–Jun. 2000, doi: 10.1002/bltj.2229.

[2] Y. Kamei, E. Shihab, B. Adams, A. E. Hassan, A. Mockus, A. Sinha, and N. Ubayashi, "A large-scale empirical study of just-in-time quality assurance," IEEE Trans. Softw. Eng., vol. 39, no. 6, pp. 757–773, Jun. 2013, doi: 10.1109/TSE.2012.70.

[3] J. Śliwerski, T. Zimmermann, and A. Zeller, "When do changes induce fixes?" in Proc. Int. Workshop Mining Softw. Repositories (MSR '05), St. Louis, MO, USA, 2005, pp. 1–5, doi: 10.1145/1083142.1083147.

[4] Y. Lyu, H. J. Kang, R. Widyasari, J. Lawall, and D. Lo, "Evaluating SZZ implementations: An empirical study on the Linux kernel," IEEE Trans. Softw. Eng., vol. 50, no. 9, pp. 2219–2239, Sep. 2024, doi: 10.1109/TSE.2024.3406718.

[5] S. Levin, "docs: add AI Coding Assistants documentation," Linux kernel, commit 78d979db6cef, 6 ene. 2026. [En línea]. Disponible: https://github.com/torvalds/linux/commit/78d979db6cef

[6] C. Brauner, "coding-assistants: simplify attribution," Linux kernel, commit 816d9992d9ed, 3 ago. 2026. [En línea]. Disponible: https://github.com/torvalds/linux/commit/816d9992d9ed

[7] S. Levin, "docs: remove Assisted-by references," Linux kernel, commit 1e035297ffda, 14 oct. 2026. [En línea]. Disponible: https://github.com/torvalds/linux/commit/1e035297ffda

[8] S. Studer, T. B. Bui, C. Drescher, A. Hanuschkin, L. Winkler, S. Peters, and K.-R. Müller, "Towards CRISP-ML(Q): A machine learning process model with quality assurance methodology," *Mach. Learn. Knowl. Extr.*, vol. 3, no. 2, pp. 392–413, 2021, doi: 10.3390/make3020020.

[9] M. Maes Bermejo, J. M. Gonzalez-Barahona, M. Gallego, and G. Robles, "A dataset of Linux kernel commits" (v3), Zenodo, 2024, doi: 10.5281/zenodo.10654193.

[10] M. J. Page *et al.*, "The PRISMA 2020 statement: An updated guideline for reporting systematic reviews," *BMJ*, vol. 372, Art. no. n71, 2021, doi: 10.1136/bmj.n71.

[11] Z. Zeng, Y. Zhang, H. Zhang, and L. Zhang, "Deep just-in-time defect prediction: How far are we?," in Proc. 30th ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA), Virtual, Denmark, 2021, pp. 427-438, doi: 10.1145/3460319.3464819.

[12] C. Ni, W. Wang, K. Yang, X. Xia, K. Liu, and D. Lo, "The best of both worlds: Integrating semantic features with expert features for defect prediction and localization," in Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE), Singapore, 2022, pp. 672-683, doi: 10.1145/3540250.3549165.

[13] Y. Liu, K. Jabre, R. Abreu, Z. J. Carmichael, V. Murali, A. Patel, J. Ge, W. Sun, C. Zhang, A. Mockus, D. Khavari, P. C. Rigby, and N. Nagappan, "A preliminary study on explaining risk of code changes using LLM-based prediction models," in Proc. 3rd ACM Int. Conf. AI-Powered Software (AIware '26), Montreal, QC, Canada, 2026, pp. 259–267, doi: 10.1145/3805760.3814916.

[14] J. Jiarpakdee, C. K. Tantithamthavorn, and J. Grundy, "Practitioners' perceptions of the goals and visual explanations of defect prediction models," in Proc. IEEE/ACM 18th Int. Conf. Mining Software Repositories (MSR), 2021, pp. 432–443, doi: 10.1109/MSR52588.2021.00055.

[15] C. Pornprasit and C. K. Tantithamthavorn, "JITLine: A simpler, better, faster, finer-grained just-in-time defect prediction," in Proc. IEEE/ACM 18th Int. Conf. Mining Softw. Repositories (MSR), 2021, pp. 369-379, doi: 10.1109/MSR52588.2021.00049.

[16] C. Pornprasit, C. Tantithamthavorn, J. Jiarpakdee, M. Fu, and P. Thongtanunam, "PyExplainer: Explaining the predictions of just-in-time defect models," in Proc. 36th IEEE/ACM Int. Conf. Automated Software Engineering (ASE), 2021, pp. 407–418, doi: 10.1109/ASE51524.2021.9678763.

[17] H. Sahar, A. A. Bangash, A. Hindle, and D. Barbosa, "IRJIT: A simple, online, information retrieval approach for just-in-time software defect prediction," Empirical Softw. Eng., vol. 29, no. 5, Art. no. 131, 2024, doi: 10.1007/s10664-024-10514-z.

[18] J. Yu, M. Fu, A. Ignatiev, C. Tantithamthavorn, and P. Stuckey, "A formal explainer for just-in-time defect predictions," ACM Trans. Softw. Eng. Methodol., vol. 33, no. 7, pp. 1–31, Aug. 2024, doi: 10.1145/3664809.

[19] X. Chen, Y. Zhao, Q. Wang, and Z. Yuan, "MULTI: Multi-objective effort-aware just-in-time software defect prediction," Inf. Softw. Technol., vol. 93, pp. 1–13, Jan. 2018, doi: 10.1016/j.infsof.2017.08.004.

[20] J. Kang, S. Kwon, D. Ryu, and J. Baik, "HASPO: Harmony search-based parameter optimization for just-in-time software defect prediction in maritime software," Appl. Sci., vol. 11, no. 5, Art. no. 2002, 2021, doi: 10.3390/app11052002.

[21] S. Mostafa, S. T. Cynthia, B. Roy, and D. Mondal, "Feature transformation for improved software bug detection and commit classification," J. Syst. Softw., vol. 219, Art. no. 112205, Jan. 2025, doi: 10.1016/j.jss.2024.112205.

[22] F. Chen, Y. Ke, X. Liu, and Q. Wei, “Multi-project just-in-time software defect prediction based on multi-task learning for mobile applications,” in Proc. IEEE Conf. Software Testing, Verification and Validation (ICST), 2025, pp. 127–137, doi: 10.1109/ICST62969.2025.10989004.

[23] S. Aftab, S. Abbas, T. M. Ghazal, M. Ahmad, H. A. Hamadi, C. Y. Yeun, and M. A. Khan, “A cloud-based software defect prediction system using data and decision-level machine learning fusion,” Mathematics, vol. 11, no. 3, Art. no. 632, 2023, doi: 10.3390/math11030632.

[24] R. Dey, J. Piri, B. Acharya, P. P. Das, V. C. Gerogiannis, and A. Kanavos, “A hybrid evolutionary fuzzy ensemble approach for accurate software defect prediction,” Mathematics, vol. 13, no. 7, Art. no. 1140, 2025, doi: 10.3390/math13071140.

[25] S. McIntosh and Y. Kamei, “Are fix-inducing changes a moving target? A longitudinal case study of just-in-time defect prediction,” IEEE Trans. Softw. Eng., vol. 44, no. 5, pp. 412–428, May 2018, doi: 10.1109/TSE.2017.2693980.

[26] M. A. Shehab, A. Hamou-Lhadj, and V. S. P. Gunda, “JITBoost: Boosting just-in-time defect prediction using Boolean combination of classifiers,” in Proc. IEEE 23rd Int. Conf. Software Quality, Reliability and Security (QRS), 2023, pp. 95–104, doi: 10.1109/QRS60937.2023.00019.

[27] Z. Chitsazian, S. Sedighian Kashi, and A. Nikanjam, “Detecting concept drift in just-in-time software defect prediction using model interpretation,” Int. J. Syst. Assur. Eng. Manag., vol. 17, no. 9, pp. 2701-2722, 2026, doi: 10.1007/s13198-025-03122-7.

[28] H. J. Patel, B. Adams, and A. E. Hassan, “Post deployment recycling of machine learning models,” Empirical Softw. Eng., vol. 29, no. 4, Art. no. 100, 2024, doi: 10.1007/s10664-024-10492-2.

[29] D. Olewicki, S. Habchi, M. Nayrolles, M. Faramarzi, S. Chandar, and B. Adams, “On the costs and benefits of adopting lifelong learning for software analytics – Empirical study on brown build and risk prediction,” in Proc. 46th Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP), Lisbon, Portugal, 2024, pp. 275–286, doi: 10.1145/3639477.3639717.

[30] D. Tian, L. Song, and X. Su, “Just-in-time quality assurance with assistance of automated defect prediction and code testing,” ACM Trans. Softw. Eng. Methodol., vol. 35, no. 9, pp. 1–46, 2026, doi: 10.1145/3779653.

[31] G. G. Cabral, L. L. Minku, E. Shihab, and S. Mujahid, “Class imbalance evolution and verification latency in just-in-time software defect prediction,” in Proc. IEEE/ACM 41st Int. Conf. Softw. Eng. (ICSE), Montreal, QC, Canada, 2019, pp. 666–676, doi: 10.1109/ICSE.2019.00076.

[32] L. Song and L. L. Minku, “A procedure to continuously evaluate predictive performance of just-in-time software defect prediction models during software development,” IEEE Trans. Softw. Eng., vol. 49, no. 2, pp. 646–666, Feb. 2023, doi: 10.1109/TSE.2022.3158831.

[33] D. Nguyen, T. Le-Cong, T. H. M. Le, M. A. Babar, and Q.-T. Huynh, “Toward realistic evaluations of just-in-time vulnerability prediction,” in Proc. IEEE Int. Conf. Software Maintenance and Evolution (ICSME), Auckland, New Zealand, 2025, pp. 1–10, doi: 10.1109/ICSME64153.2025.00023.

[34] I. Mazloomzadeh, M. M. Morovati, and F. Khomh, “How do AI coding agents contribute to software development? An empirical study of agentic pull requests,” arXiv:2607.21832, Jul. 2026, doi: 10.48550/arXiv.2607.21832.

[35] S. Herbold, A. Trautsch, F. Trautsch, and B. Ledel, “Problems with SZZ and features: An empirical study of the state of practice of defect prediction data collection,” Empir. Softw. Eng., vol. 27, no. 2, Art. no. 42, 2022, doi: 10.1007/s10664-021-10092-4.

[36] R. C. Veras, G. G. Cabral, and A. L. I. de Oliveira, “Semantic SZZ: Mitigating the impact of misclassified corrective changes in just-in-time software defect prediction,” in Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC), 2025, pp. 769-774, doi: 10.1109/SMC58881.2025.11342537.

[37] D. Falessi, A. Ahluwalia, and M. Di Penta, “The impact of dormant defects on defect prediction: A study of 19 Apache projects,” ACM Trans. Softw. Eng. Methodol., vol. 31, no. 1, pp. 1–26, Jan. 2022, doi: 10.1145/3467895.

[38] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” The Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, Oct. 2001, doi: 10.1214/aos/1013203451.

[39] E. H. Ruspini, “A new approach to clustering,” Information and Control, vol. 15, no. 1, pp. 22–32, Jul. 1969, doi: 10.1016/S0019-9958(69)90591-9.

[40] F. Herrera, “Genetic fuzzy systems: taxonomy, current research trends and prospects,” Evolutionary Intelligence, vol. 1, no. 1, pp. 27–46, Mar. 2008, doi: 10.1007/s12065-007-0001-5.

[41] H. Ishibuchi, T. Nakashima, and T. Murata, “Three-objective genetics-based machine learning for linguistic rule extraction,” Information Sciences, vol. 136, no. 1–4, pp. 109–133, Aug. 2001, doi: 10.1016/S0020-0255(01)00144-X.

[42] T. Saito and M. Rehmsmeier, “The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets,” PLOS ONE, vol. 10, no. 3, Art. no. e0118432, Mar. 2015, doi: 10.1371/journal.pone.0118432.

[43] T. G. Dietterich, “Approximate statistical tests for comparing supervised classification learning algorithms,” Neural Computation, vol. 10, no. 7, pp. 1895–1923, Oct. 1998, doi: 10.1162/089976698300017197.

[44] J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” Journal of Machine Learning Research, vol. 7, no. 1, pp. 1–30, 2006.